

# EventHub: Full Stack Community Event Management Platform

Detailed project brief generated from the project data.

Software Developer (Full Stack Junior)

CANDIDATE  
Rahul Sharma

DURATION  
120 days

DIFFICULTY  
junior

TYPE  
fullstack

TECHNOLOGY STACK

- JavaScriptTypeScriptReactNext.jsNode.jsExpress.jsLaravel (PHP)Tailwind CSSMongoDB
- MongoDBJWTGitDockerVercel



# Project Overview

A concise view of the problem, purpose, ownership and expected outcome.

PROJECT SUMMARY

A web platform where organizers can create events, attendees can browse and register, and admins can manage users and generate PDF tickets. The project evaluates API design, security, performance, testing and cloud deployment using the candidate's existing stack.

ROLE

Software Developer (Full Stack Junior)

TYPE

fullstack

DIFFICULTY

junior

DURATION

120 days

FOCUS

api\_development, problem\_solving, security, cloud, sys...

CANDIDATE

Rahul Sharma

## Objectives

- Deliver a responsive Next.js UI for event browsing and management.
- Expose a secure REST API with proper error handling.
- Create a Laravel service that returns PDF tickets on demand.
- Automate CI/CD pipelines for testing and deployment.
- Document architecture, API contracts and deployment steps.

## Candidate Details

Candidate information available in the project record.

Name Rahul Sharma

## Technology Stack

Languages, frameworks, services and tools relevant to implementation.

JavaScript

TypeScript

React

Next.js

Node.js

Express.js

Laravel (PHP)

Tailwind CSS

MongoDB

JWT

Git

Docker

Vercel



## Functional Requirements

### Requirement 1

Users can register, log in and receive a JWT.

### Requirement 2

Organizers can create, edit, and delete their events.

### Requirement 3

Attendees can browse events, filter by date/location, and register.

### Requirement 4

System issues a unique ticket per registration and stores it.

### Requirement 5

Admins can view all users, events and generate usage reports.

### Requirement 6

Attendees can download a PDF ticket generated by the Laravel service.

## Non-Functional Requirements

### Requirement 1

All passwords must be hashed with bcrypt.

### Requirement 2

API responses must be returned within 300 /ms for 95 /% of requests.



**Requirement 3**

Input data must be validated server side to prevent injection.

**Requirement 4**

The application must be accessible (WCAG /2.1 AA) on desktop and mobile.

**Requirement 5**

Logs must be written to stdout for Docker container monitoring.

## System Modules

Major functional areas that make up the project.

**01****Authentication**

JWT login, role assignment and token refresh.

- Register
- Login
- Password hashing
- Role based middleware

**02****User Management**

Admin CRUD for users and role changes.

- List users
- Edit profile
- Assign roles
- Deactivate account



03

## Event Management

Organizer CRUD for events with image upload.

- Create event
- Edit event
- Delete event
- Publish/unpublish

04

## Ticketing

Registration flow, ticket issuance and storage.

- Register for event
- Generate ticket ID
- Store ticket
- Cancel registration

05

## PDF Generation

Laravel microservice that returns a PDF ticket.

- Receive ticket ID
- Render PDF with QR code
- Return file stream

06

## Search & Filtering

Frontend search bar and backend query support.

- Keyword search
- Date range filter
- Location filter
- Pagination



07

### Admin Dashboard

Analytics view for event attendance and revenue.

- Overview charts
- Export CSV
- User activity log
- Event performance metrics

08

### CI/CD & Deployment

Dockerized backend, Vercel frontend, automated tests.

- GitHub Actions workflow
- Docker build
- Vercel preview deploy
- Smoke tests

## Implementation Roadmap

Recommended delivery sequence based on the project data.

PHASE 1

### Foundation

Set up repo, CI pipeline and project skeleton.

Tasks: Initialize monorepo with frontend and backend folders • Configure ESLint/Prettier for JS/TS • Create Dockerfile for Express API • Set up GitHub Actions for lint a...  
Deliverables: Git repo • CI config • Dockerfile



## PHASE 2

**Auth & User API**

Implement secure JWT authentication and user CRUD.

Tasks: Design User schema (MongoDB) • Implement register & login routes • Add bcrypt password hashing • Create role based middleware

Deliverables: Auth API • User model • Auth test suite

## PHASE 3

**Event & Ticket Core**

Build event CRUD and ticket issuance logic.

Tasks: Define Event and Ticket schemas • Create event CRUD routes with validation • Implement registration endpoint (Promise.all for parallel checks) • Store tickets wit...

Deliverables: Event API • Ticket model • Integration tests

## PHASE 4

**PDF Ticket Service**

Integrate Laravel microservice for PDF generation.

Tasks: scaffold Laravel project (Docker container) • Create endpoint /ticket/{id} returning PDF • Use dompdf to render ticket with QR code • Add API client in Node to ca...

Deliverables: Laravel PDF service • API client wrapper • E2E PDF test



PHASE 5

Admin Dashboard & Reporting

Provide analytics and CSV export for admins.

Tasks: Design admin pages with Next.js and Tailwind • Create reporting API (attendance, revenue) • Implement CSV export endpoint • Add role based access control to admin...  
Deliverables: Admin UI • Reporting API • CSV export

PHASE 6

Quality, Security & Production Deploy

Finalize testing, harden security and launch.

Tasks: Add rate limiting middleware • Run OWASP ZAP scan and fix findings • Write performance benchmark scripts • Configure Vercel production and Docker production images  
Deliverables: Security report • Performance results • Production deployment

API Reference

Key endpoints exposed by the project.

| METHOD | ENDPOINT                 | DESCRIPTION                               |
|--------|--------------------------|-------------------------------------------|
| POST   | /api/auth/register       | Create new user                           |
| POST   | /api/auth/login          | Authenticate and return JWT               |
| GET    | /api/events              | List public events with filters           |
| POST   | /api/events              | Organizer creates event                   |
| POST   | /api/events/:id/register | Attendee registers and receives ticket ID |
| GET    | /api/admin/reports       | Admin fetches analytics data              |

Database Design

Entities, collections or tables required by the project.



**Entity 1**

\_id  
name  
email  
passwordHash  
role  
createdAt

**Entity 2**

\_id  
title  
description  
date  
location  
organizerId  
isPublic

**Entity 3**

\_id  
eventId  
userId  
ticketCode  
issuedAt  
status

**Entity 4**

name  
permissions



# Evaluation Criteria

Suggested areas for assessing project completion and quality.

|                                                           |     |
|-----------------------------------------------------------|-----|
| Correctness                                               | 25% |
| <div><div></div></div>                                    |     |
| All functional requirements work as specified             |     |
| Code Quality                                              | 15% |
| <div><div></div></div>                                    |     |
| Readable, modular code with linting passed                |     |
| Security                                                  | 15% |
| <div><div></div></div>                                    |     |
| Proper auth, input validation, no sensitive data leaks    |     |
| Testing                                                   | 15% |
| <div><div></div></div>                                    |     |
| Coverage "e 80 /% and includes security/performance tests |     |
| Performance                                               | 15% |
| <div><div></div></div>                                    |     |
| API response times meet non functional targets            |     |
| Documentation & Deployment                                | 15% |
| <div><div></div></div>                                    |     |
| Clear README, API docs, CI/CD pipeline functional         |     |

# Expected Deliverables

Concrete outputs that should exist when the project is completed.

- 1

Deliverable 1

Complete source code (frontend & backend) in GitHub
- 2

Deliverable 2

README with setup, run and deployment instructions
- 3

Deliverable 3

Swagger/OpenAPI spec for all endpoints
- 4

Deliverable 4

MongoDB schema diagram
- 5

Deliverable 5

Automated test suite



**Deliverable 6**

6

Docker compose file and Vercel project link

## Student Fit

Information connecting the project requirements with the candidate profile.

Rahul already knows JavaScript/TypeScript, React, Next.js, Node, Express, MongoDB, JWT and role based auth, which cover the core of the platform. The project adds a modest Laravel service for PDF generation and Docker/Vercel deployment, providing useful new learning while staying within a junior scope. The 120 day timeline allows phased development, testing and security hardening, matching his skill level and giving ample opportunity to demonstrate problem solving, system design and full stack implementation.

## Requirements

- Implement role based JWT authentication (admin, organizer, attendee).
- Provide CRUD APIs for events and tickets with validation.
- Generate PDF tickets via a Laravel microservice.
- Deploy the full stack on Vercel (frontend) and Docker (backend).
- Include unit, integration and security tests covering all core flows.

## Functional Requirements

- Users can register, log in and receive a JWT.
- Organizers can create, edit, and delete their events.
- Attendees can browse events, filter by date/location, and register.
- System issues a unique ticket per registration and stores it.
- Admins can view all users, events and generate usage reports.
- Attendees can download a PDF ticket generated by the Laravel service.

## Non Functional Requirements

- All passwords must be hashed with bcrypt.
- API responses must be returned within 300 /ms for 95 % of requests.
- Input data must be validated server side to prevent injection.
- The application must be accessible (WCAG /2.1 AA) on desktop and mobile.
- Logs must be written to stdout for Docker container monitoring.

## Testing Plan

- Unit tests for service layer (Jest)
- Integration tests for API routes (Supertest)
- E2E tests for registration flow (Cypress)
- Auth tests for JWT validation and role enforcement
- Performance benchmark for event list endpoint
- Security scan using OWASP ZAP

## Student Id

6ab2576039d4e38828d96a4e



# Status

active

## Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.



# Contents

Sections included in this project specification.

**01**

Project Overview

[p. 2](#)**02**

Candidate Details

[p. 2](#)**03**

Technology Stack

[p. 2](#)**04**

Functional Requirements

[p. 3](#)**05**

Non-Functional Requirements

[p. 3](#)**06**

System Modules

[p. 4](#)**07**

Implementation Roadmap

[p. 6](#)

**09**

Database Design

[p. 8](#)**10**

Evaluation Criteria

[p. 10](#)**11**

Expected Deliverables

[p. 10](#)**12**

Student Fit

[p. 11](#)**13**

Requirements

[p. 11](#)**14**

Functional Requirements

[p. 11](#)**15**

Non Functional Requirements

**16**

Testing Plan

[p. 11](#)

**17**

Student Id

[p. 11](#)

**18**

Status

[p. 12](#)